HS PF

Programmieren in Python

Python-Grundlagen: Numpy Teil 3

Prof. Dr.-Ing. Stefan Kray

Daten mit NumPy finden und ändern (1)

- Häufig sucht man bestimmte Daten mit Numpy und möchte diese ändern
- Dazu gibt es mehrere Möglichkeiten
- Beispiel:

```
data=np.array([[1, 2, 3],
               [4, 5, 6],
               [7, 8, 9]])
```

Mit Vergleichsoperatoren:

```
data>=5
array([[False, False, False],
       [False,  True,  True],
       [ True,  True,  True]])

data[data>=5]=0
array([[1, 2, 3],
       [4, 0, 0],
       [0, 0, 0]])
```

Daten mit NumPy finden und ändern (2)

```
data=np.array([[1, 2, 3],
               [4, np.nan, 6],
               [7, 8, 9]],dtype=np.float)
```

- **Mit speziellen Operatoren**

```
np.isnan(data)
array([[False, False, False],
       [False,  True, False],
       [False, False, False]])
```

Weitere: `np.isinf`, `np.isreal`, `np.iscomplex`

Daten mit NumPy finden und ändern (3)

```
data=np.array([[1, 2, 3],
               [4, 5, 6],
               [7, 8, 9]])
```

- **Mit np.where: Liefert Indizes zurück**

```
y,x=np.where(data>=7)
#y: array([2, 2, 2], dtype=int64)
#x: array([0, 1, 2], dtype=int64)

data[y,x]=42
#array([[ 1.,  2.,  3.],
#       [ 4.,  5.,  6.],
#       [42., 42., 42.]])
```

Daten mit NumPy finden und ändern (4)

```
data=np.array([[1, 2, 3],
               [4, 5, 6],
               [7, 8, 9]])
```

- **Mit np.where auch verkette Anfragen möglich:**

```
y,x=np.where(((data>=5) & (data<=7)) | (data==1))
#y: array([0, 1, 1, 2], dtype=int64)
#x: array([0, 1, 2, 0], dtype=int64)
data[y,x] #array([1., 5., 6., 7.])
```

Häufig: Prüfen auf Werte ungleich Null:

```
y,x=np.where(data!=0)
```

Kurzform: `y,x=data.nonzero()`

Arrays zusammenfügen (1)

- Arrays kann man vertikal oder horizontal zusammenfügen
- Vertikal:

```
img1=np.zeros((3,3))
img2=np.ones((3,3))
img3=np.vstack((img1,img2))
#array([[0., 0., 0.],
#       [0., 0., 0.],
#       [0., 0., 0.],
#       [1., 1., 1.],
#       [1., 1., 1.],
#       [1., 1., 1.]])
```

Arrays zusammenfügen (2)

- Arrays kann man vertikal oder horizontal zusammenfügen
- horizontal:

```
img1=np.zeros((3,3))
img2=np.ones((3,3))
img3=np.hstack((img1,img2))
#array([[0., 0., 0., 1., 1., 1.],
#       [0., 0., 0., 1., 1., 1.],
#       [0., 0., 0., 1., 1., 1.]])
```

Arrays zusammenfügen (3)

- Arrays kann man vertikal oder horizontal zusammenfügen
- „mehrdimensional“, z.B. für Farbkanäle:

```
img1=np.zeros((3,3)))
img2=np.ones((3,3))
img3=np.dstack((img1,img2))
#array([[[0., 1.],
#        [0., 1.],
#        [0., 1.]],
#
#       [[0., 1.],
#        [0., 1.],
#        [0., 1.]],
#
#       [[0., 1.],
#        [0., 1.],
#        [0., 1.]]])
#img3.shape: (3,3,2)
```

Arrays trennen

- Umkehrung des Zusammenfügens: `np.split`

```
x = np.arange(9.0)
np.split(x, 3)
[array([0.,  1.,  2.]), array([3.,  4.,  5.]), array([6.,  7.,  8.])]
```

- **Nicht äquidistante Teilung:**

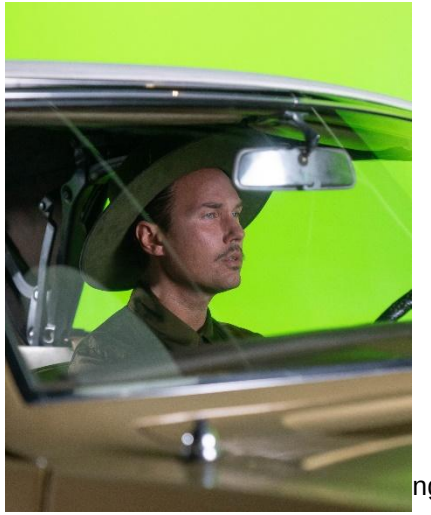
```
x = np.arange(8.0)
np.split(x, [3, 5, 6, 10]) #Split-Positionen
[array([0.,  1.,  2.]),
 array([3.,  4.]),
 array([5.]),
 array([6.,  7.]),
 array([], dtype=float64)]
```

Weitere Kommandos: `np.dsplit`, `np.hsplit`, `np.vsplit`

Aufgabe rgb.py

```
import matplotlib.image as mpimg
img = mpimg.imread('testbild.png').copy()
```

- Laden Sie die Datei „greenscreen.jpg“ als Variable img
- Analysieren Sie den Aufbau des Bildes (Größe, Farbkanäle, Wertebereich)
- Laden Sie zudem die Datei „background.jpg“
- Analysieren Sie den Aufbau des Bildes (Größe, Farbkanäle, Wertebereich)
- Ersetzen Sie nun die grüne Farbe in img durch das Hintergrundbild.
  - Dazu erzeugen Sie in neues Bild aus dem alten: img2=img.copy()
  - Und verändern img2 so, dass die grüne Farbe durch das Hintergrundbild ersetzt wird.
- Sie können Ihr neues Bild mit der Matplotlib: mpimg.imsave("test.jpg",img2) abspeichern.







Padding (1)

- Häufig muss man Arrays „am Rand“ mit Werten auffüllen

```
a=np.array([1,2,3,4,5])
np.pad(a, (2, 3))
#array([0, 0, 1, 2, 3, 4, 5, 0, 0, 0])

np.pad(a, (2, 3), 'linear_ramp')
#array([0, 0, 1, 2, 3, 4, 5, 3, 1, 0])

np.pad(a, (2, 3), 'constant', constant_values=(4, 6))
#array([4, 4, 1, 2, 3, 4, 5, 6, 6, 6])
```

Padding (2)

- Auch mehrdimensional möglich

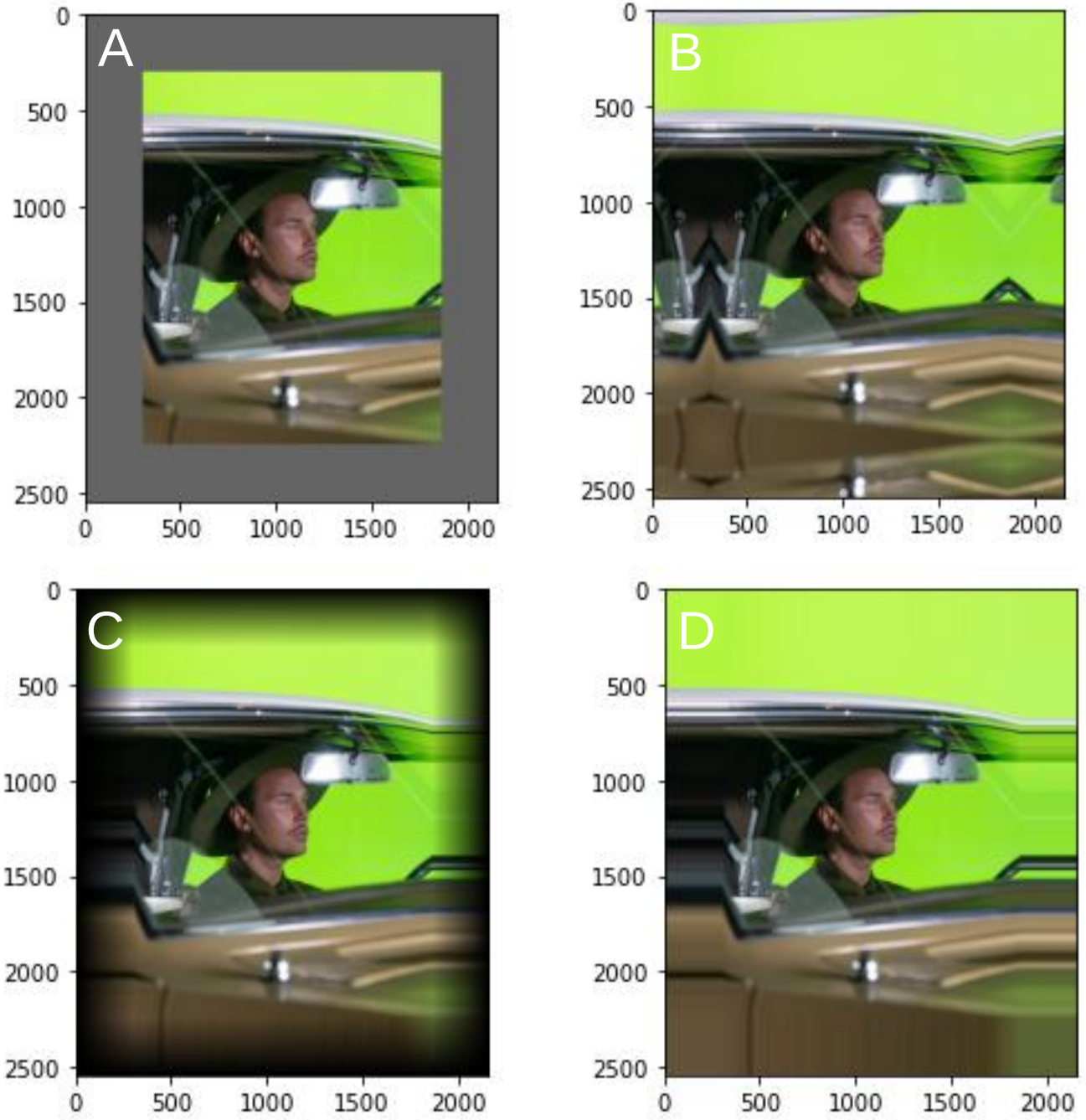
```
A: plt.imshow(np.pad(img,((300,300),(300,300),(0,0)),
mode="constant",constant_values=(100,100)))

B:

plt.imshow(np.pad(img,((300,300),(300,300),(0,0)),
mode="reflect"))

C: plt.imshow(np.pad(img,((300,300),(300,300),(0,0)),
mode="linear_ramp"))

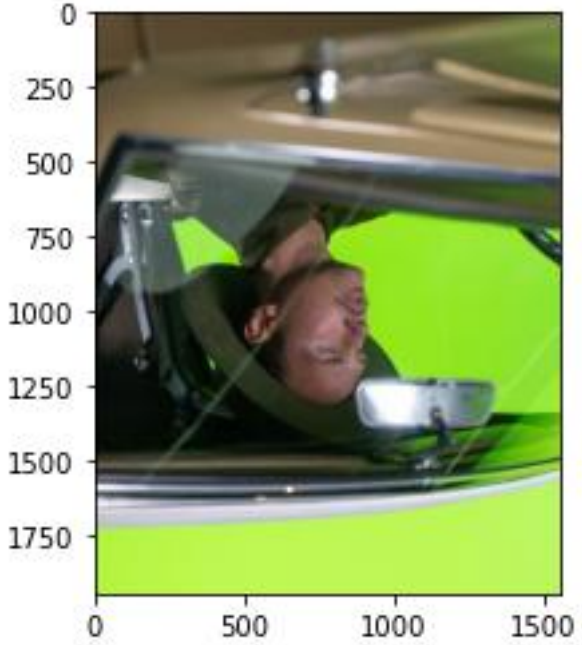

D: plt.imshow(np.pad(img,((300,300),(300,300),(0,0)),
mode="edge"))
```



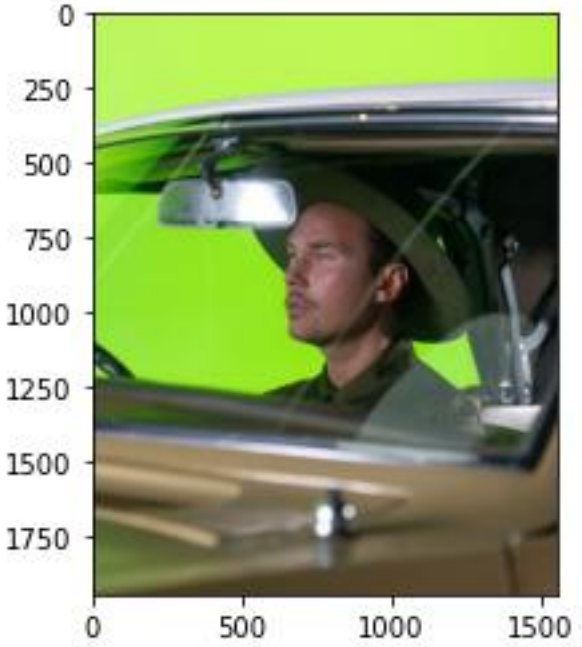
Spiegeln

- Mehrdimensionale Arrays lassen sich mit np.flip spiegeln

```
plt.imshow(np.flip(img,axis=0))
plt.imshow(np.flipud(img))
```



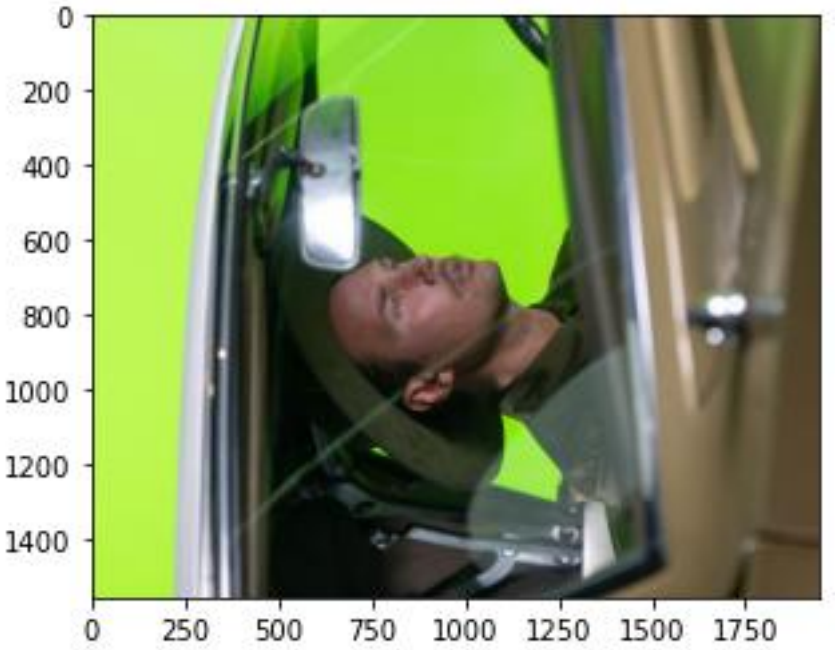

```
plt.imshow(np.flip(img,axis=1))
plt.imshow(np.fliplr(img))
```



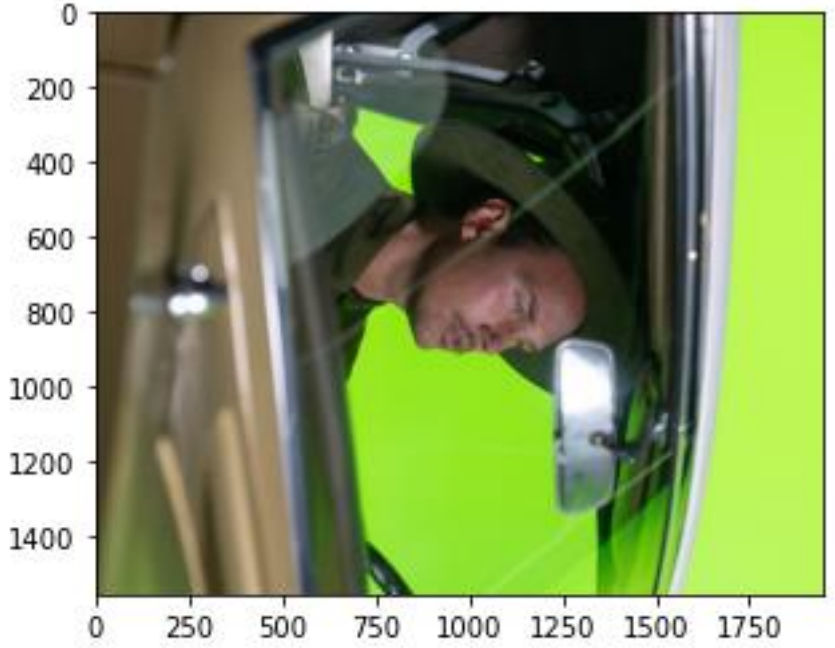
Rot90

- Mehrdimensionale Arrays lassen sich mit np.rot90 drehen

```
plt.imshow(np.rot90(img))
```

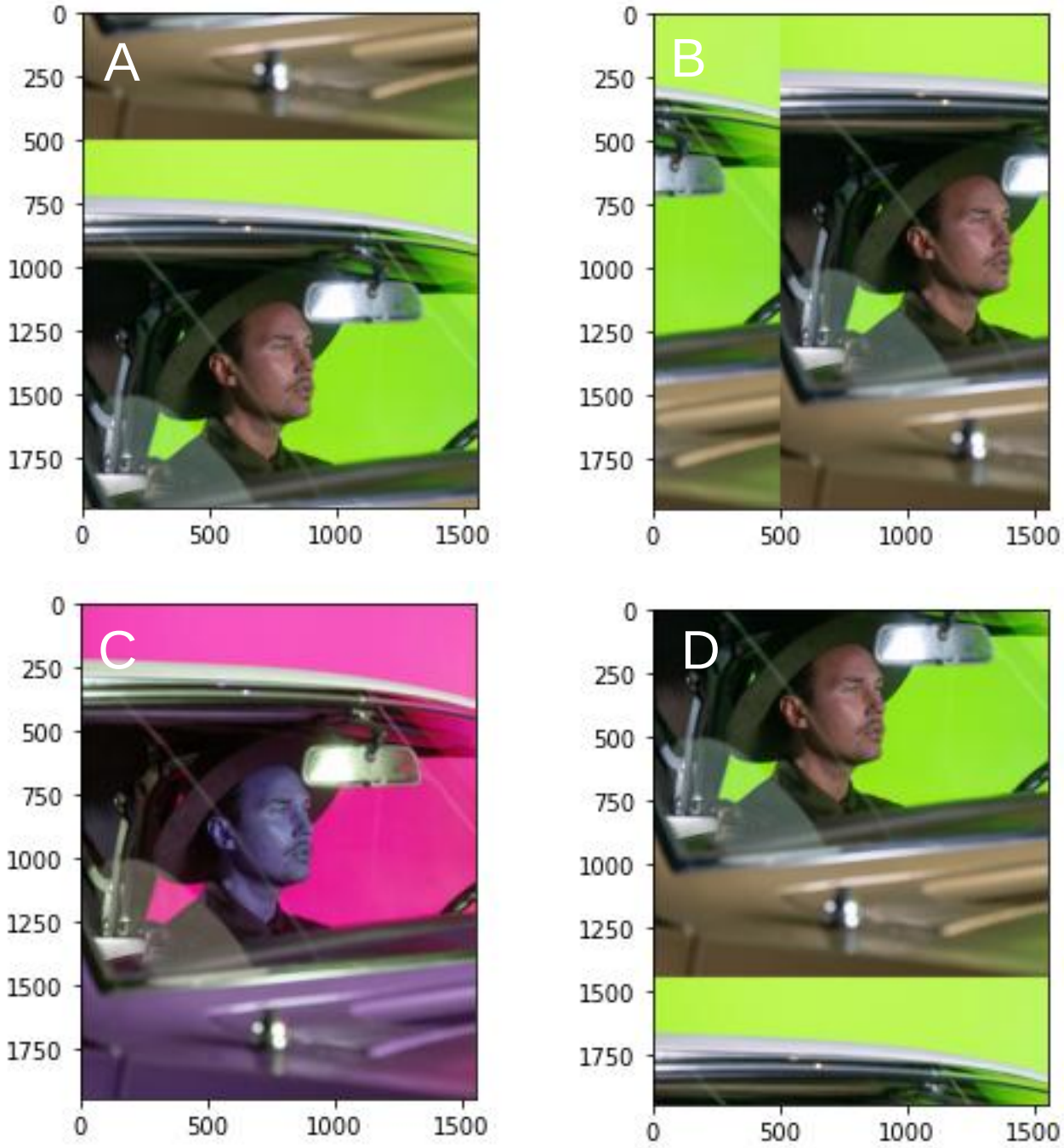



```
plt.imshow(np.rot90(img,k=3)) #k: Anzahl Drehungen
mit axes-Parameter wird Rotationsachse angegeben
```



Rollieren

- Numpy.roll rolliert um eine Achse:
- A:`plt.imshow(np.roll(img,500,axis=0))`
- B:`plt.imshow(np.roll(img,500,axis=1))`
- C:`plt.imshow(np.roll(img,500,axis=2))`
- D:`plt.imshow(np.roll(img,-500,axis=0))`
-



Aufgabe logo.py

- Erzeugen Sie folgendes Bild:
  - Laden Sie den Hintergrund „background.jpg“
  - Laden Sie das Python-Logo „logo.bmp“
  - Transformieren Sie das Logo entsprechend
  - Überlegen Sie, wie Sie die Spiegelung erzeugen können
  - Farben Sie die Spiegelung dunkler
  - Speichern Sie Ihr Ergebnis als Bild ab

